左光明 - AI Agent / Web 全栈工程师

> 目标不是“会一点前后端”，而是能把 AI、权限、支付、实时通信、异步任务和部署链路收束成一个可运行、可维护、可扩展的企业级系统。

基本信息

姓名：左光明

学历：本科 · 武汉文理学院 · 计算机科学与技术 · 2026 届

电话：15671628271

邮箱：2093146753@qq.com

方向：AI Agent 工程师 / Web 全栈工程师 / 企业级后台系统开发

地点：武汉 / 全国 / 远程均可

期望薪资：8K 起

在线系统：zgm2003.cn

技术社区：Linux.do 三级用户

个人定位

我不是按“前端、后端、某一种语言”来定义自己的开发者。我更关注的是：**一个系统的边界是否清楚、数据流是否可信、异步任务是否可恢复、权限模型是否可演进、AI Agent 是否能被观测和治理、部署后是否真的能稳定跑起来**。

目前我最核心的证明，是独立设计并上线了一套企业级 AI Admin 系统：前端、后端、数据库、权限、Agent、SSE、WebSocket、支付、队列、桌面端更新和线上部署都由我自己完成。

技术能力

AI Agent / LLM 工程化

- 能把 LLM 能力落成可管理的工程系统：模型接入、Agent 编排、工具调用、流式输出、运行审计、超时取消和错误暴露。
- 设计过 Agent、Model、Tool、Prompt、Conversation、Message、Run、Run Step 等运行模型，能把一次 AI 调用拆成可观测状态机。
- 熟悉 OpenAI-compatible 接口、SSE 流式对话、历史消息拼装、工具执行记录、结果截断和失败恢复。
- 关注 Tool Calling 安全边界：HTTPS 白名单、SSRF 防护、只读 SQL、写操作拦截、自动 LIMIT 和敏感结果控制。
- 理解 RAG 的检索、召回、重排、上下文拼装链路，能为后续知识库和 Agent Step 扩展预留模型。
- 开发过 AI Make 本地 UI 生成 Skill / CLI，把提示词、视觉规范、任务拆分、Agent handoff 和 Review Gate 组织成可执行工作流。

前端 / 客户端工程

- Vue 3 / React、TypeScript、Vite、Element Plus、Ant Design、Pinia、Zustand、Vue Router、React Router、Vue I18n、Vant、uni-app。
- 熟悉后台前端工程化：统一请求封装、ApiEnvelope 解包、401 刷新队列、动态路由、按钮权限、SSE 严格解析、WebSocket 单例连接。
- 能拆分前端基础设施边界：HTTP、Auth Session、Stream、Realtime、Router、Table / CRUD composable、权限快照和 i18n 文案边界。
- 有 Electron / Tauri 桌面端经验，处理窗口能力、IPC / Bridge、Preload、版本检测、更新清单、安装包构建和 CSP 安全策略。

后端 / 数据 / 自动化

- PHP 8.1+、Laravel 8、Webman / Workerman、Eloquent、MySQL、Redis、Redis Queue、GatewayWorker、Crontab、PHPUnit。
- 能写 Python 自动化脚本，用于批量数据处理、接口调试、素材/文件处理、运营数据清洗和 AI 工具链辅助，不把重复脏活留给人工。
- 熟悉 Node.js / JavaScript ESM 脚本开发，能做 CLI、代码生成、任务拆分、文件扫描和本地工程工具封装。
- 坚持 Controller -> Module -> Dep -> Model 分层：Controller 只转发，Module 只编排，Dep 负责数据访问，Model 只做表映射。
- 熟悉认证权限：Access / Refresh Token、Sanctum、Token Hash + Pepper、Redis Session、单端登录、平台/设备/IP 绑定、RBAC 菜单/路由/按钮权限。
- 熟悉异步与后台任务：Redis Queue、Laravel Job、定时任务、AI 超时检测、通知调度、支付关单/同步/履约/对账任务。

部署 / 交付

- 独立完成域名、HTTPS、Nginx 反代、Webman 多端口服务、MySQL、Redis、COS 静态资源与更新清单配置。
- 能区分 API、SSE、WebSocket、桌面端本地能力和外部渠道回调的运行形态，并按协议特性做隔离。
- 关注契约测试、接口边界和工程约束，不靠前端空数组、空对象、静默 catch 掩盖后端错误。

工作经历

小药药医药科技有限公司 · 前端开发

时间：2025.10.27 - 至今

- 从 0 搭建公司 SaaS 商家端前端工程，覆盖 Web 独立运行与 Electron Desktop 打包运行两条链路。
- 负责登录、租户/门店选择、总部/门店工作区切换、权限菜单、会话恢复、统一请求和 CRUD 基础设施等核心工程能力。
- 参与荷叶问诊后台、移动端 APP、问诊内核 H5 三端迭代，覆盖远程审方、视频问诊、合理用药审查、长处方/慢病规则、移动推送与跨端跳转等医疗问诊核心链路。
- 在 Figma Make 生成代码质量不稳定、UI 基础组件边界不清的约束下，逐步收口页面结构、组件边界和交互规范，避免设计稿代码直接污染业务层。

江苏麦影网络科技有限公司 · 全栈开发

时间：2024.06 - 2025.06

- 参与麦小图 VCMS 内容管理 / 发布系统迭代，覆盖视频发布、内容管理、达人运营、内容运营、后期运营、素材中心、数据中心和运营任务等核心模块。
- 参与 Electron + Vue 3 桌面端建设，通过 IPC、BrowserWindow / BrowserView、Preload 和本地存储能力承载多平台授权、内容发布、账号检测和运营辅助流程。
- 对接淘宝、京东、小红书、B 站、支付宝、得物、快手、拼多多等内容/电商渠道，处理 OAuth、Cookie 注入、渠道发布回调、账号池和平台数据同步等复杂链路。
- 参与 AI 内容生产与视频任务流转能力，覆盖商品/素材管理、AIGC 文案生成、视频混剪、字幕/语音/封面、审核状态和队列任务处理。
- 支撑运营后台高频能力：团队/角色/权限、批量编辑、Excel 导入导出、数据统计、内容审核和任务状态流转，提升内容运营批处理效率。

项目经历

智澜·TS 企业级 AI Admin 系统

角色：个人项目 / 独立设计与开发 / 已上线 **在线地址**：https://zgm2003.cn **技术栈**：Vue 3.5、TypeScript、Vite 8、Element Plus、Pinia、Vue Router、Vue I18n、Tauri 2、PHP 8.1+、Webman / Workerman、Eloquent、MySQL 8.4、Redis、Redis Queue、GatewayWorker、NeuronAI、Yansongda Pay、腾讯云 COS / TTS、阿里云 AIGC / TTS。

项目概述

智澜·TS 是一套面向企业后台管理与 AI 能力集成的全栈 Admin 系统。它不是单纯的 CRUD Demo，而是一个包含 **认证权限、动态菜单、AI Agent、流式对话、IM 聊天、支付钱包、订单履约、上传存储、通知任务、导出任务、系统日志、Tauri 桌面端更新** 的完整工程。

这个项目真正体现的是我的工程能力：我能把复杂系统拆成可维护边界，把 AI 能力接入业务系统，把异步任务、实时通信、权限模型和部署链路做成闭环。

1. AI Agent 运行系统

- 设计 Agent / Model / Tool / Prompt / Conversation / Message / Run / Step 数据模型，让一次 AI 调用从“黑盒请求”变成可追踪、可审计、可取消、可超时治理的运行过程。
- 支持多模型 Provider 和 OpenAI-compatible 接口，面向不同 Agent 绑定模型、系统提示词和工具能力。
- 后端通过独立 SSE 服务输出 `conversation`、`run`、`content`、`tool_call`、`tool_result`、`done`、`error`、`canceled` 等事件；前端按事件驱动更新 UI，而不是拼接假状态。
- 前端对 SSE 做严格解析：malformed payload、缺失终止事件、运行失败都会直接暴露，避免“看起来完成了，实际状态坏了”的假成功。

2. Agent Tool 安全治理

- 实现 Internal Tool、HTTPS 白名单 Tool、只读 SQL Tool 三类工具执行器。
- HTTPS 工具执行前做域名/IP 检查，拒绝 IP 直连和内网地址访问，降低 SSRF 风险。
- SQL 工具限制只允许 SELECT，拒绝 INSERT / UPDATE / DELETE / DROP 等写关键字，并自动追加 LIMIT，避免 Agent 直接变成数据库破坏入口。
- 工具调用链路保留执行记录和结果截断，兼顾可观测性与安全边界。

3. 企业级后台架构

- 后端坚持 Controller -> Module -> Dep -> Model 分层，避免接口、业务、查询、表映射混在一起。
- Controller 只做路由入口，Module 负责编排验证、事务和业务流程，Dep 负责 Eloquent 查询与缓存读写，Model 只保留表映射和 casts。
- Service 承载跨模块能力，Lib 封装第三方 SDK，保证 AI、支付、上传、TTS 等外部能力不会污染业务层。
- 当前后端规模：**43 个 Controller、48 个 Module、46 个 Dep、47 个 Model、21 个 Service、10 个 Redis Queue 消费者**。

4. 认证、权限与动态路由

- 实现 Access / Refresh Token 机制，Token 明文不落库，仅存 Hash；结合 Pepper、Redis Session、过期校验和降级 DB 查询。
- 支持单端登录指针、平台/设备/IP 绑定策略，降低 Token 被复用后的风险。
- RBAC 权限按平台输出菜单树、动态路由和按钮权限码，前端通过 `userStore.can(code)` 控制按钮级权限。
- 动态路由只接收后端 `view_key`，前端通过 `import.meta.glob` 精确解析视图组件，不做路径猜测和历史字段兜底。

5. 前端基础设施与强契约

- 封装 Axios 客户端，统一处理 ApiEnvelope 解包、错误提示、认证 Header 和 401 刷新队列。
- 拆分 `useTable` 与 `useCrudTable`：前者只管列表与分页，后者才处理搜索、删除、批量删除、状态切换等 CRUD 编排。
- WebSocket 客户端封装为单例连接，支持绑定用户、自动重连、ping/pong 和消息总线分发。
- 对前后端契约采取“后端错就暴露”的策略，不用空数组、空对象、静默 catch 去掩盖接口问题。

6. 支付、钱包与订单闭环

- 接入 Yansongda Pay，完成微信/支付宝支付相关能力封装。
- 实现充值订单、支付流水、钱包入账、订单履约、支付回调、对账任务和定时补偿。
- 使用 RedisLock 控制重复提交和回调并发，避免支付链路重复入账或重复履约。
- 将履约、关单、同步、对账放入后台任务，减少主请求链路阻塞。

7. 商品 AI 工作台

- 围绕电商商品构建 OCR -> AI 口播生成 -> TTS 合成 -> SRT 下载的异步流水线。
- 支持商品采集、图片选择、OCR 识别、Agent 生成卖点/口播词、语音合成和字幕文件下载。
- 使用 Redis Queue 承载 OCR、AI、TTS 等耗时任务，让用户操作链路保持即时响应。

8. 桌面端与线上部署

- 使用 Tauri 2 封装线上 Admin，配置窗口能力、进程退出、版本检测、NSIS 打包、COS 更新清单和 CSP。
- 独立完成服务器部署、域名解析、HTTPS、Nginx 反代、MySQL / Redis 配置和前端发布。
- 线上服务拆分为 API `8787`、SSE `8788`、WebSocket `7272`，按协议特性分别反代。

SaaS 商家端 Web / Desktop 一体化前端

角色：公司项目 / 前端架构与核心开发 / 从 0 搭建 **技术栈**：React 19、TypeScript、Vite、Electron、Ant Design、TanStack React Query、Zustand、React Router、Axios、Zod、Tailwind CSS、Electron Builder。

项目概述

该项目是面向药店/商家的 SaaS 管理端，既支持浏览器 Web 运行，也支持 Electron 桌面端本地运行和打包发布。我负责从工程初始化到核心链路落地：运行时识别、接口基址配置、登录恢复、门店/总部工作区、权限菜单、统一请求、

状态管理和通用 CRUD 页面能力。

核心工作

- 搭建 Web / Desktop 双运行链路：Web 端使用 Vite 独立构建，桌面端通过 Electron 承载本地运行、后端 ready bridge、窗口能力和安装包构建。
- 设计运行时初始化与接口客户端配置，区分 Web、Desktop、本地后端、业务 API 等不同基础地址，避免页面代码到处判断运行环境。
- 落地登录、Token 恢复、门店选择/申请、总部/门店工作区切换、权限菜单和动态路由入口，让用户会话和业务工作区状态可恢复、可切换。
- 封装 Zustand session/users 状态、Axios 请求客户端、统一错误处理、权限快照和 i18n 文案边界，减少页面层重复状态和重复判断。
- 在 Figma Make 生成代码质量不稳定的现实约束下，把页面代码逐步收口到 Ant Design 与 `components/ui` 基础组件，沉淀 Dialog、Search、Table、Column Settings、CRUD Hook 等可复用能力。
- 坚持强契约开发：前端不靠猜字段、空兜底和静默 catch 掩盖接口问题，优先让协议错误暴露出来，再按真实契约修正。

荷叶问诊医药 SaaS 三端协同系统

角色：公司项目 / 核心前端开发 **技术栈**：Vue 3、TypeScript、Vite、Element Plus、Vant、uni-app、Pinia、Axios / luch-request、腾讯云 IM / TRTC、阿里云移动推送、Aegis。

项目概述

该项目覆盖 PC 管理后台、跨端移动 APP 和问诊内核 H5。核心业务不是普通页面 CRUD，而是围绕互联网医院与药店问诊场景，把患者、医生、药师、商家、处方、审方、视频通话、移动推送和合规规则串成可用链路。

核心亮点

- 后台端承接远程审方、药师工作台、处方记录、问诊数据、GSP 商品资料和合理用药规则配置，处理动态路由、租户 `tenant-id`、Token 刷新队列和全局错误提示。
- 移动端基于 uni-app 支持 H5、Android、iOS、微信小程序和鸿蒙配置，接入阿里云推送、腾讯 IM / TRTC，处理处方待审、视频来电、订单通知、权限申请和跨端页面跳转。
- 问诊内核 H5 使用 Vue 3 + Vant 承载患者问诊、医生接诊、药师审方、第三方问诊流转、商品提交、套餐购买和消息中心等移动端业务。
- 在第三方问诊链路中梳理详情拉取、合理用药审查、药品/诊断归一化、审查弹窗拦截和后续流转，避免把医疗规则判断散落在页面事件里。
- 排查过长处方/慢病仍提示超量的问题，定位到“慢病病情选择”和“后台慢病目录药品配置”是两层不同规则，问题根因不在前端状态，而在规则目录匹配。
- 能在医疗问诊这种强状态、强合规、多角色、多端协同的系统里做问题定位：先追请求 payload、路由来源、推送上下文和后端配置，再判断前端是否需要改。

AI Make 本地 UIUX Patch Compiler

角色：个人项目 / 产品设计与独立开发 / Codex Skill + npm CLI **技术栈**：Node.js、JavaScript ESM、Codex Skill、Prompt Engineering、React、TypeScript、Tailwind CSS、Multi-Agent Workflow。

项目概述

AI Make 是我围绕 Figma Make 工作流沉淀的本地开发者 UI 生成 Skill。它不是网页 IDE，也不直接调用模型 API，而是把一句 UI 需求编译成 **visual brief、page composition blueprint、ui-spec、prompt-pack、agent handoff、任**

务拆分和 review gates，再交给 Codex / Claude Code / Cursor 等本地编码 Agent 在现有项目里生成可审查、可验证、可合并的前端 patch。

核心工作

- 设计从自然语言 UI 需求到本地代码 patch 的编译链路：先确定视觉方向和首屏结构，再生成规格、提示词包、Agent 交接文档和验证要求。
- 实现 `ai-make` CLI 与 Codex Skill 入口，支持读取目标项目上下文、生成运行目录、拆分任务、输出单任务 Agent prompt，并记录本轮 patch 的 review gate。
- 把视觉质量放在代码质量之前：通过 `visual-brief` 和 `page composition blueprint` 约束首屏框架、hero、指标节奏、证据区、行动区和细节层，避免生成“能跑但像模板”的页面。
- 强约束生成边界：不猜后端字段、不添加 fallback 字段、不绕过项目架构、不擅自引入新 UI 库、不生成巨大单文件页面。
- 支持本地多 Agent 协作思路，通过任务依赖和 write set 限定每个 Agent 的编辑范围，降低并行开发时的冲突和风格漂移。
- 沉淀 `visual-quality`、`spec-compliance`、`react-quality`、`verification` 四类 Review Gate，让 AI 生成 UI 具备可审查的交付标准，而不是一次性 Prompt 产物。

教育经历

武汉文理学院 · 计算机科学与技术 · 本科

毕业时间：2026.06

我的优势

- **不是只会写页面**：能从页面走到接口、数据库、队列、权限、Agent、部署和桌面端。
- **不是只会调模型**：能把 LLM 接成 Agent 运行系统，考虑工具安全、运行审计、超时、取消和前端流式体验。
- **不是只会堆功能**：更关注模块边界、前后端契约、数据一致性和系统长期维护成本。
- **不被语言栈限制**：PHP、TypeScript、JavaScript、Python 都只是工具，真正重要的是架构、边界和交付。